

GROCERY STORE MANAGEMENT SYSTEM

Final Project Report

Submitted By:

Name: S M Mahamudul Haque; **Student ID:** 2677527

Name: Shahriya Alam; **Student ID:** 2677517

Name: Hosen Aiyub; **Student ID:** 2677530

Name: Hossain Md Miraj; **Student ID:** 2677529

Course Title: Master Software design I

Date of Submission: 15th June, 2026

ABSTRACT

This report describes the design, implementation, and evaluation of a Grocery Store Management System (GSMS), a Python/Flask web application that supports product and inventory management, order processing, and user authentication for a grocery store. The system provides an admin dashboard, CRUD interfaces for products and units of measure, and order creation with automatic total calculation. We discuss the project goals, requirements, system architecture (a three-tier web app), design artifacts (use-case, ER, and sequence diagrams), implementation details (modules, data models, code structure), sample data analysis, testing and deployment on Render, and considerations of security, usability, and project management. Appendices include installation/run instructions and key code excerpts. The system meets high-priority functional requirements such as viewing, adding, editing, and deleting products, user login, and order processing, along with non-functional requirements related to performance and reliability.

Contents

ABSTRACT.....	i
CHAPTER 1	
INTRODUCTION	1
1.1 Background of the Study	1
1.2 Problem Statement	1
1.3 Project Objectives	1
1.4 Scope of the Project	1
1.5 Significance of the Project	1
CHAPTER 2	
LITERATURE REVIEW	2
2.1 Introduction.....	2
2.2 Grocery Store Management Systems.....	2
2.3 Inventory Management Systems.....	2
2.4 Web-Based Retail Management Applications	2
2.5 Research Gap	2
2.6 Summary	2
CHAPTER 3	
REQUIREMENTS ANALYSIS	3
3.1 Existing System Analysis	3
3.2 Proposed System	3
3.3 Functional Requirements	3
3.4 Non-Functional Requirements	3
3.5 User Requirements.....	3
3.6 Use Case Requirements	4
3.7 Requirements Traceability	7
CHAPTER 4	
SYSTEM DESIGN	9
4.1 System Architecture.....	9
4.2 Context Diagram	10
4.3 Use Case Diagram.....	10
4.4 Database Design.....	11
4.5 Sequence Diagram	12

CHAPTER 5	
MOCKUP	14
CHAPTER 6	
BUSINESS MODEL	15
CHAPTER 7	
AI TOOLS USED	16
7.1 AI Tools Utilized and Their Contributions	16
CHAPTER 8	
IMPLEMENTATION	17
8.1 Development Environment	17
8.2 Technology Stack.....	17
8.3 Key Modules and Responsibilities.....	17
8.4 Sample Data Analysis	17
CHAPTER 9	
TESTING AND EVALUATION	18
9.1 Testing Strategy	18
9.2 Test Cases and Results.....	18
9.3 Evaluation	18
CHAPTER 10	
DEPLOYMENT AND SECURITY	19
10.1 Deployment on Render	19
10.2 Authentication and Password Security	19
10.3 Input Validation and Session Handling	19
10.4 Usability and Accessibility	19
10.5 Limitations and Security Considerations	19
CHAPTER 11	
PROJECT MANAGEMENT	20
11.1 Project Planning.....	20
11.2 Team Responsibilities	20
11.3 Challenges Faced	20
11.4 Gantt Chart Analysis.....	20
11.5 Screenshot of the Requirements Traceability Matrix (RTM)	21
CHAPTER 12	

CONCLUSION AND FUTURE WORK	23
12.1 Conclusion	23
12.2 Future Enhancements.....	23
REFERENCES	24
APPENDICES	25

CHAPTER 1

INTRODUCTION

1.1 Background of the Study

The Grocery Store Management System (GSMS) was developed to help a small grocery store manage products, units of measure, and customer orders through a single web-based interface. The project addresses the common operational need for accurate inventory tracking, quicker order entry, and easier administrative control.

1.2 Problem Statement

Manual grocery store record keeping is slow, error-prone, and difficult to maintain as the number of products and transactions grows. A lightweight web-based solution is needed to centralize product management, automate order total calculation, and maintain consistent records.

1.3 Project Objectives

- Design a web application for managing grocery products and order records.
- Provide secure user authentication for administrative access.
- Support product CRUD operations and order creation with automatic total calculation.
- Store data in a structured and maintainable format for reliable reuse.
- Deploy the application on a public cloud platform for easy access.

1.4 Scope of the Project

The scope of the project covers the management of products, units of measure, users, and orders. The implemented system focuses on essential store operations rather than advanced enterprise functions such as procurement automation, supplier analytics, or barcode integration.

1.5 Significance of the Project

The project demonstrates how a small-scale business process can be digitized using a modern Python web stack. It also serves as a practical academic exercise in software analysis, design, deployment, and documentation.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

This chapter summarizes the technical context of grocery and retail management applications. The project sits at the intersection of inventory management, order processing, and web application development.

2.2 Grocery Store Management Systems

Grocery store management systems are designed to help small retailers keep track of products, quantities, and customer transactions. A practical system typically combines a product catalog, ordering flow, and administrative dashboard.

2.3 Inventory Management Systems

Inventory management software records stock levels and updates them as sales occur. This makes it easier to maintain accurate product information and reduces the risk of mismatched records.

2.4 Web-Based Retail Management Applications

Web-based applications are a natural fit for small retail systems because they can be accessed through a browser and deployed centrally. Flask is suitable for such applications because it is lightweight, flexible, and easy to organize into routes, templates, and data access modules.

2.5 Research Gap

The project focuses on a simple and understandable implementation rather than a large enterprise platform. Its academic value lies in showing how essential management features can be implemented, tested, and deployed with limited complexity.

2.6 Summary

The literature context supports the choice of a browser-based architecture, structured data management, and a clear administrative workflow for the GSMS project.

CHAPTER 3

REQUIREMENTS ANALYSIS

3.1 Existing System Analysis

In a manual workflow, product details are often stored in notebooks or spreadsheets, and order calculation is performed by hand. Such approaches can work for small volumes but become difficult to maintain and verify over time.

3.2 Proposed System

The proposed GSMS centralizes the store's core data inside a web application. The system allows a logged-in administrator to manage products, view inventory records, and create orders with computed totals.

3.3 Functional Requirements

- View products on the dashboard.
- Add a new product with name, unit of measure, and price.
- Edit existing product information.
- Delete a product from the listing.
- Create a customer order by selecting products and quantities.
- Automatically calculate the total for each order.
- Store order details persistently.
- View past orders and order history.
- Require user authentication before accessing management pages.

3.4 Non-Functional Requirements

- Performance: common actions should respond quickly for small data volumes.
- Reliability: stored data should remain consistent and recoverable.
- Usability: the interface should be simple and easy to navigate.
- Maintainability: the code should be modular and easy to extend.
- Security: login, password hashing, and validation should be enforced.
- Portability: the application should run on any system with Python [3] and a supported web server.

3.5 User Requirements

The Grocery Store Management System is designed for an authenticated store operator or administrator who is responsible for managing products and orders through a web interface. Based on the implemented source code, the system does not use separate user roles such as admin, cashier, or manager. Instead, it provides the same operational privileges to any registered user who successfully logs in.

Administrator role

The administrator role represents the authenticated user who accesses the system after login. This user is responsible for performing the main business operations supported by the application, including product management, order creation, and viewing order history. The administrator also has access to the dashboard, which provides an overview of total orders, total revenue, total products, and recent transactions.

The administrator is expected to:

- log in using a valid email and password;
- register a new account when necessary;
- manage product records by adding, editing, and deleting products;
- create new customer orders by selecting products and quantities;
- review past orders and order details from the dashboard;
- log out after completing the session.

System access permissions

The system uses session-based authentication to control access to protected pages and API endpoints. Access permissions are defined as follows:

- **Unauthenticated users**
 - Can access the login page and registration page.
 - Cannot access the dashboard, product management page, or order page.
 - Are redirected to the login page if they try to open a protected page directly.
- **Authenticated users**
 - Can access the dashboard.
 - Can view, add, edit, and delete products.
 - Can create new orders and view saved order history.
 - Can access authenticated API endpoints such as product retrieval, UOM retrieval, order saving, and order listing.
 - Can log out to end the current session.
- **Session control**
 - After successful login, the user's name and email are stored in the session.
 - Logout clears the session and removes access to protected features.
 - The protected routes in the backend enforce authentication before allowing access.

3.6 Use Case Requirements

The system supports the following main use cases based on the implemented Flask [2] routes, HTML pages, and JavaScript logic.

Use Case 1: Login

Actor: Administrator / Authorized User

Goal: Gain access to the system dashboard and management pages.

Pre-condition: The user must have a valid registered account.

Main flow:

1. The user opens the login page.
2. The user enters an email address and password.
3. The system validates the credentials against the stored user records in users.json.
4. If the credentials are correct, the system creates a session for the user.
5. The user is redirected to the dashboard.

Alternative flow:

- If the email or password is invalid, the system rejects the login and displays an error message.
- If the user is not authenticated and tries to access a protected page, the system redirects them to the login page.

Post-condition: The user is authenticated and can access protected pages until logout.

Use Case 2: Manage Products

Actor: Administrator / Authorized User

Goal: Maintain product records in the system.

Pre-condition: The user must be logged in.

Main flow:

1. The user opens the Product Management page.
2. The system loads the current list of products from the backend.
3. The user can add a new product by entering the product name, unit of measure, and price per unit.
4. The user can edit an existing product by selecting it from the list and updating its details.
5. The user can delete a product when it is no longer needed.
6. After each operation, the product list is refreshed automatically.

Alternative flow:

- If any required field is empty, the system rejects the submission and requests complete input.
- If the user confirms deletion, the selected product is removed from the list.

Post-condition: The product records are updated and stored persistently in the JSON data store.

Use Case 3: Create Orders

Actor: Administrator / Authorized User

Goal: Create and save a customer order.

Pre-condition: The user must be logged in, and products must already exist in the system.

Main flow:

1. The user opens the New Order page.
2. The user enters the customer name.
3. The user selects one or more products from the product list.
4. The user enters the quantity for each selected product.
5. The system automatically calculates the total for each item and the grand total for the order.
6. The user submits the order.
7. The backend validates the order data and saves the order details.
8. The system confirms that the order has been saved successfully.

Alternative flow:

- If the customer name is missing, the system shows a validation message.
- If no product is selected or quantity is invalid, the system rejects the order and requests correct input.
- If products are still loading, the system prevents submission until the data is ready.

Post-condition: The order is stored in store.json and becomes available in order history and dashboard views.

Use Case 4: View Order History

Actor: Administrator / Authorized User

Goal: Review previous orders and inspect order details.

Pre-condition: The user must be logged in.

Main flow:

1. The user opens the dashboard.
2. The system loads all stored orders and displays them in the recent orders table.
3. The dashboard shows summary information such as total orders, total revenue, active products, and the latest order.
4. The user clicks the View button for a specific order.

5. The system displays the order details, including customer name, date, product names, unit prices, quantities, and line totals.

Alternative flow:

- If no orders exist, the system displays a message indicating that no orders are available yet.
- If an order has no line items, the system shows a message that no order items were found.

Post-condition: The user can review previously saved orders and inspect their detailed contents.

3.7 Requirements Traceability

The Requirements Traceability Matrix (RTM) establishes a relationship between the project requirements, system design components, implementation modules, and testing status. It ensures that every requirement has been addressed during the development process and can be traced from specification through implementation and verification.

Table 3.1 Functional Requirements Traceability

Requirement ID	Requirement Description	Design Component Module /	Implemented In	Verification Method	Status
FR001	View Products	Product Management Module	server.py, products_dao.py, manage-product.html	Product List Display Test	Implemented
FR002	Add Product	Product Management Module	server.py, products_dao.py, data_store.py	Add Product Test	Implemented
FR003	Edit Product	Product Management Module	server.py, products_dao.py, data_store.py	Edit Product Test	Implemented
FR004	Delete Product	Product Management Module	server.py, products_dao.py, data_store.py	Delete Product Test	Implemented
FR005	Manage Categories / Units of Measure	UOM Management Module	uom_dao.py, data_store.py	UOM Retrieval Test	Partially Implemented
FR006	Create Customer Order	Order Management Module	server.py, orders_dao.py, order.html	Order Creation Test	Implemented

Requirement ID	Requirement Description	Design Component / Module	Implemented In	Verification Method	Status
FR007	Calculate Order Total	Order Processing Logic	orders_dao.py, data_store.py	Order Total Validation Test	Implemented
FR008	Store Order Details	Order Persistence Module	data_store.py, store.json	Data Storage Verification Test	Implemented
FR009	View Orders	Order Management Module	server.py, orders_dao.py, order.html	Order Retrieval Test	Implemented
FR010	User Authentication	Authentication Module	server.py, users.json, login.html, register.html	Login and Registration Test	Implemented

Table 3.2 *Non-Functional Requirements Traceability*

Requirement ID	Requirement Description	Implementation Approach	Verification Method	Status
NFR001	Performance	Lightweight Flask architecture and JSON storage	Response Time Testing	Satisfied
NFR002	Reliability	Structured data persistence and validation checks	Functional Testing	Satisfied
NFR003	Usability	Bootstrap-based responsive interface	User Acceptance Testing	Satisfied
NFR004	Maintainability	Modular architecture using DAO pattern	Code Review	Satisfied
NFR005	Security	Password hashing, session management, authentication controls	Security Testing	Satisfied
NFR006	Availability	Deployment through Render cloud platform	Deployment Testing	Satisfied

Tables 3.1 and 3.2 illustrate the traceability of functional and non-functional requirements throughout the development lifecycle. Each requirement has been mapped to the corresponding design component, implementation module, and testing activity to ensure complete coverage and verification.

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

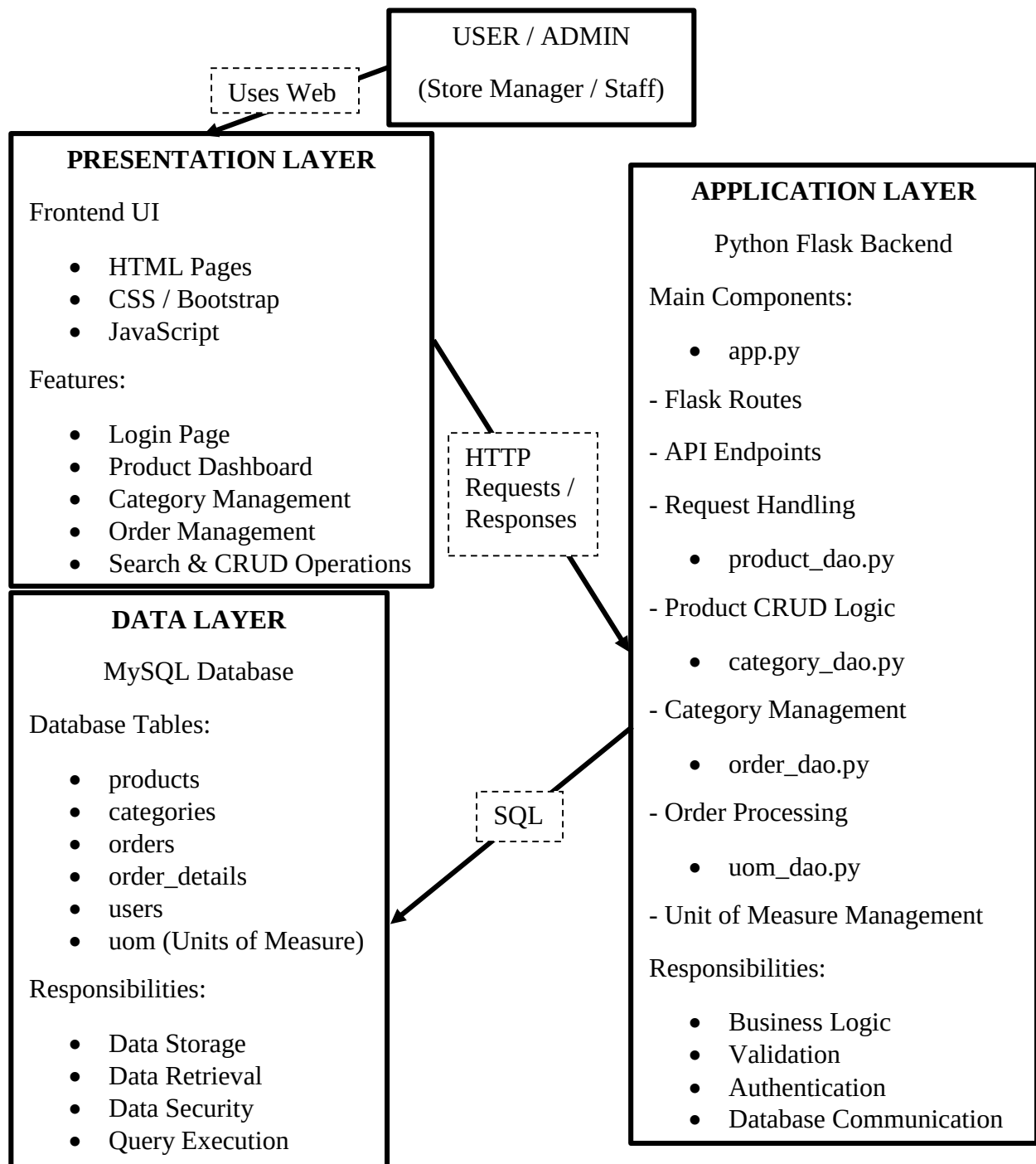


Figure 4.1 System Architecture Diagram

GSMS follows a three-tier web architecture: a browser-based presentation layer, a Flask application layer, and a data storage layer. The interface sends requests to Flask routes, the server applies business logic, and the data is persisted in local storage.

4.2 Context Diagram

The primary external actor is the store administrator. The administrator uses the web interface to log in, manage products, create orders, and review records.

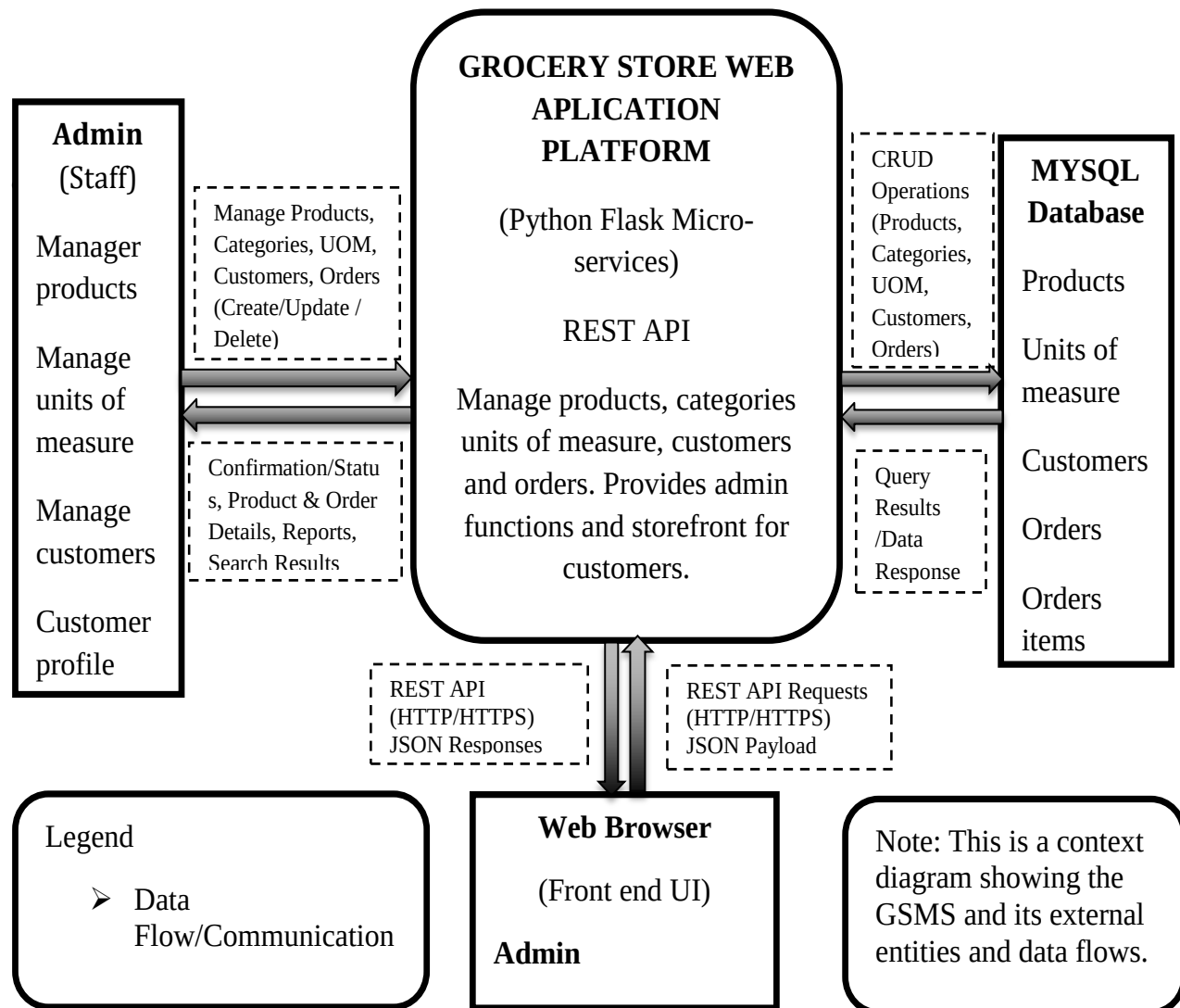


Figure 4.2 Context Diagram

4.3 Use Case Diagram

The use-case model includes login, logout, product management, order creation, and order review. These use cases represent the core administrative tasks supported by the application.

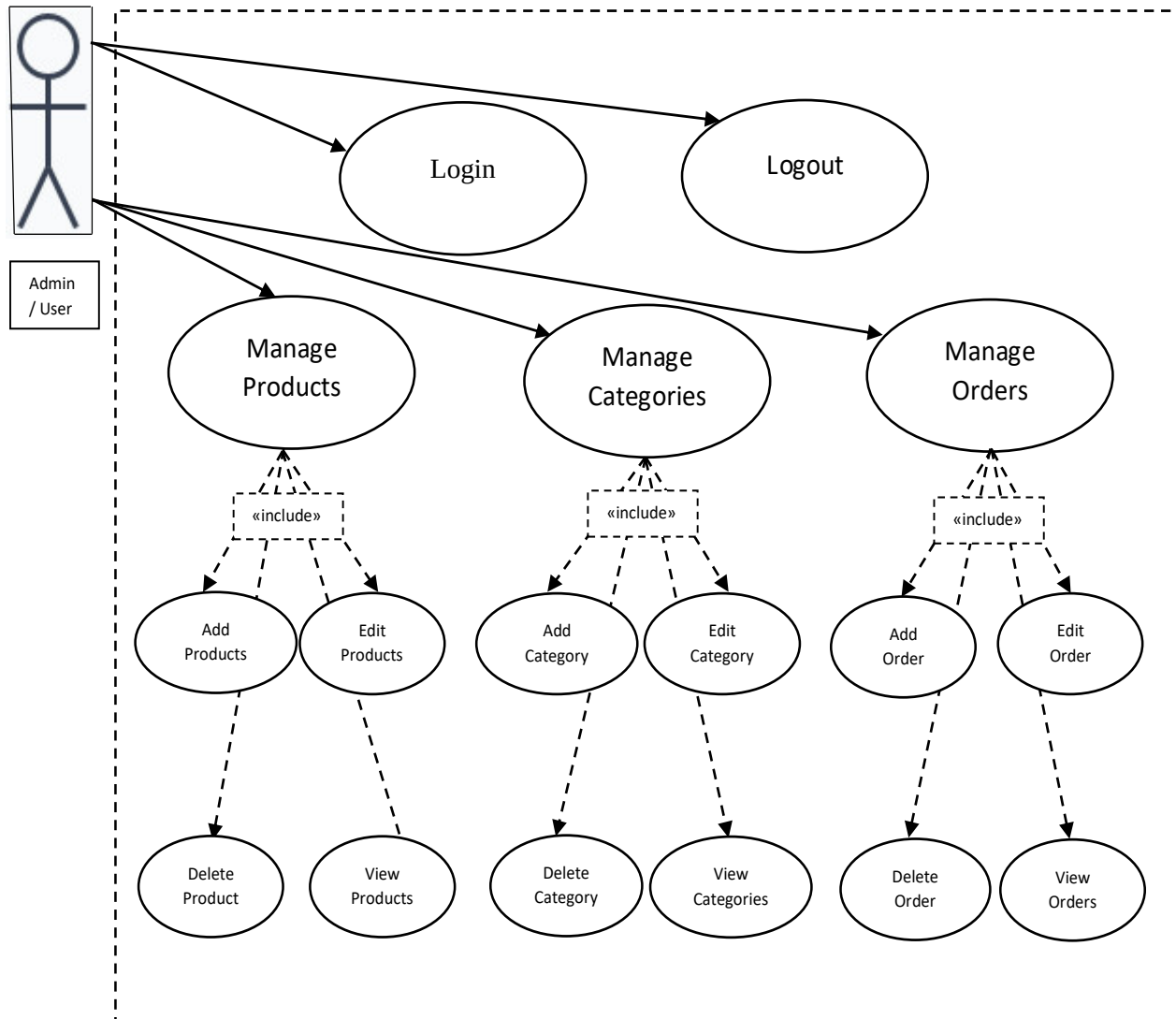


Figure 4.3 Use Case Diagram

4.4 Database Design

The conceptual design includes units of measure, products, orders, and order details. The provided SQL schema models the same entities with relational tables, while the implemented system stores equivalent structures in JSON files.

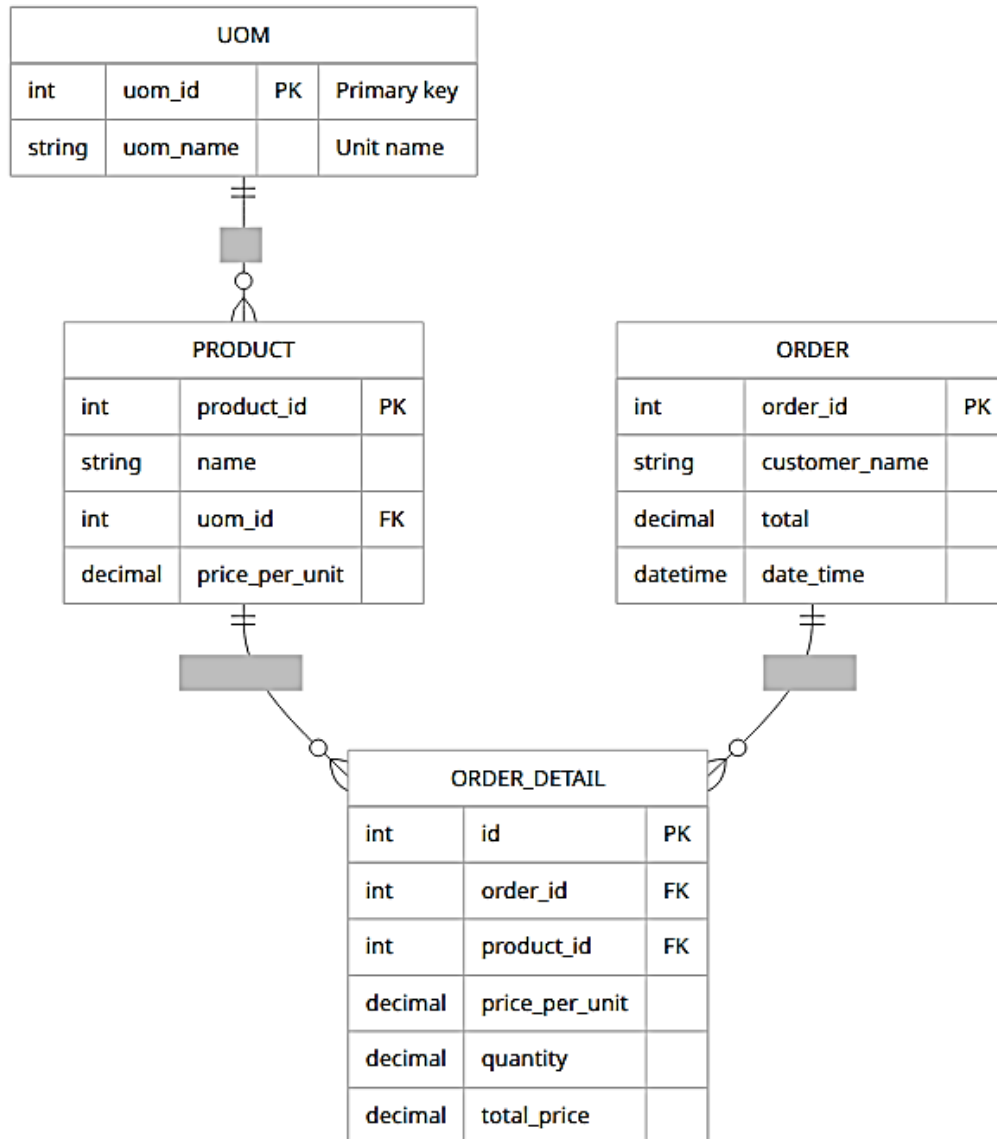


Figure 4.4 Entity-Relationship diagram for GSMS data model (UOM = unit of measure, ORDER_DETAIL holds line items of orders).

4.5 Sequence Diagram

The order creation sequence begins when the administrator submits the order form. The backend validates the input, calculates the total, and stores the record before refreshing the order list.

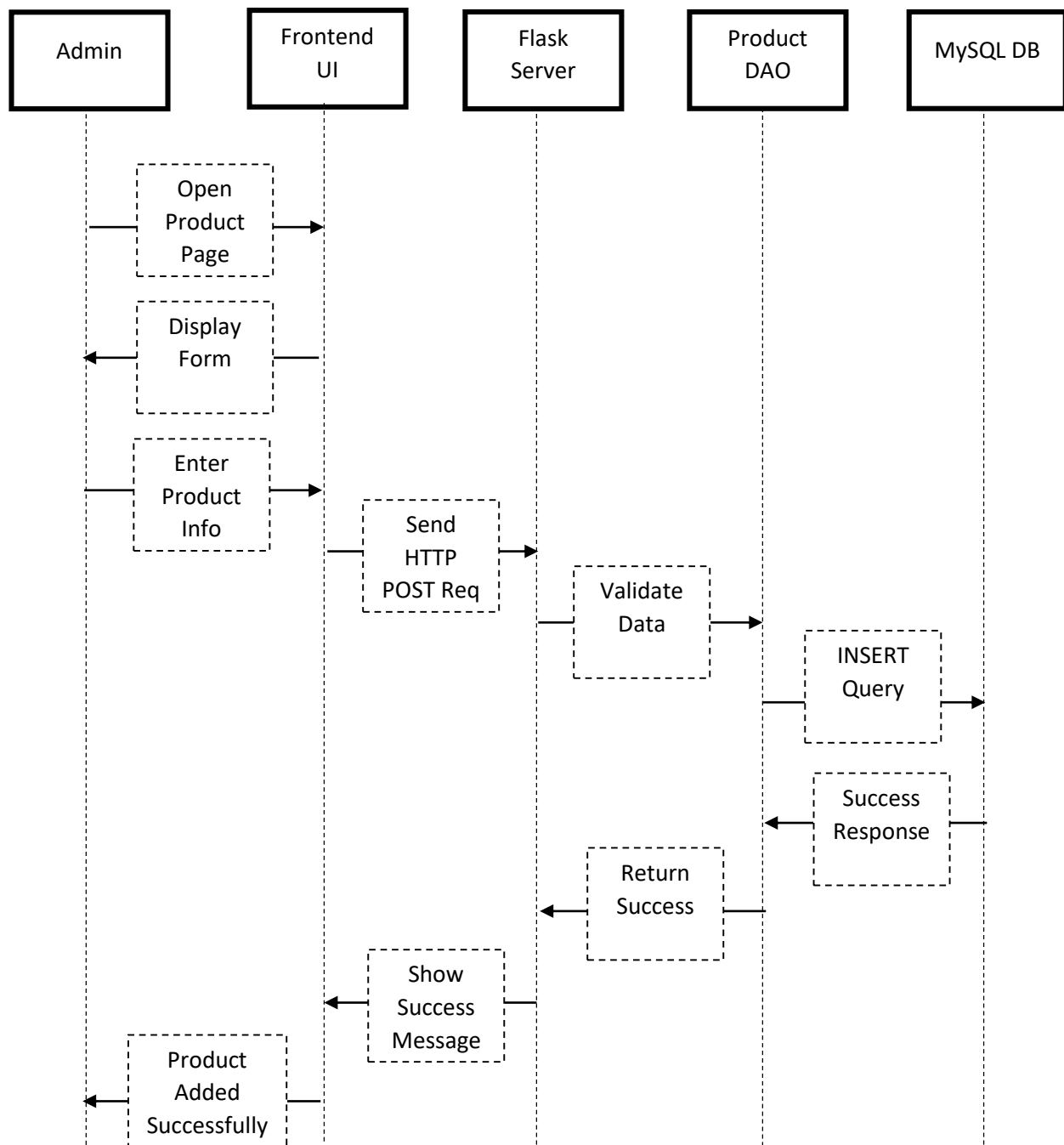


Figure 4.5 Sequence Diagram

CHAPTER 5

MOCKUP

The detailed mockup presents the visual structure of the Grocery Store Management System as it appears to the user after the application is run. It includes the main dashboard page and the product management page, which are the two primary interfaces used for day-to-day store administration.

The dashboard mockup shows the overall summary of the system, including the sidebar navigation, top bar, key summary cards, and the recent management section. This page is designed to give the user a quick overview of the store's activity and provide access to the main functions of the application.

The mockup was created to represent the actual user interface of the project and to demonstrate how the system is intended to look and operate in a browser environment. It helps explain the layout, navigation flow, and main functions of the application in a clear and practical way.

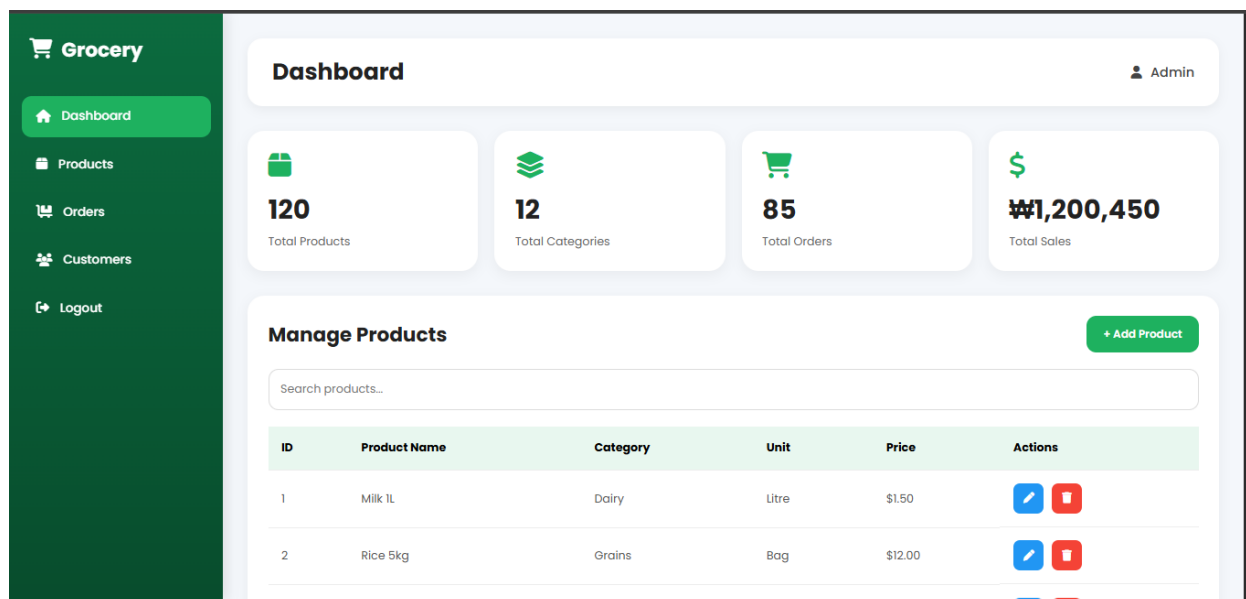


Figure 5.1 Dashboard Mockup

CHAPTER 6

BUSINESS MODEL

The Grocery Store Management System can be monetized as a small business software solution for grocery stores, mini markets, and local retail shops that want to move from manual record keeping to a simple digital system. The main value of the project is that it reduces time spent on product tracking, order entry, and administrative tasks while improving accuracy and organization.

One possible business model is a one-time license fee for small shops that want to install the system on their own computer or server. In this model, the customer pays once to use the software and receives the core features such as product management, order management, login security, and dashboard reporting. This approach is suitable for very small businesses that prefer a low-cost purchase instead of ongoing payments.

A second option is a subscription-based software service. In this model, the system can be hosted online and shops pay monthly or yearly to access it through a web browser. Subscription pricing is useful because it creates recurring income and can include technical support, backup management, updates, and new feature releases. This model also benefits business owners because they do not need to manage the software infrastructure themselves.

The project can also generate income through customization and support services. For example, a client may request a custom dashboard, additional reports, barcode integration, invoice printing, supplier management, or user-role control. These features can be offered as paid add-ons. Training services for staff, setup assistance, and maintenance contracts can also become part of the monetization strategy.

Another revenue source is a premium version of the system. The basic version may include product and order management, while the premium version could include analytics, sales forecasting, export to Excel/PDF, automatic alerts for low stock, and multi-user access control. This allows the software to serve both small and growing businesses.

Overall, the project has strong commercial potential because grocery stores always need inventory control, sales tracking, and easy administrative tools. The system solves a real business problem and can be packaged either as a low-cost local installation or as a cloud-based subscription service. Its simplicity, practicality, and usefulness make it suitable for monetization in small retail markets.

CHAPTER 7

AI TOOLS USED

7.1 AI Tools Utilized and Their Contributions

Several Artificial Intelligence (AI) tools were utilized during the development and documentation of the Grocery Store Management System (GSMS). These tools were used to support software development activities, improve the quality of technical writing, and assist in the creation of project diagrams and visual materials. All generated content was reviewed, verified, and modified where necessary to ensure accuracy and consistency with the project requirements.

ChatGPT (OpenAI) [6] was used throughout the preparation of the project report. It assisted in improving the clarity, grammar, structure, and academic writing style of numerous sections, including the Abstract, Introduction, System Analysis and Requirements, Deployment and Security, Conclusion and Future Work, Business Model, and Mockup documentation. ChatGPT was also used to refine explanations, improve readability, and ensure consistency across the report.

Claude (Anthropic) [7] was used to provide support for selected programming tasks during development. It assisted with code review, debugging suggestions, implementation ideas, and code refinement for certain components of the application.

Gemini (Google) [8] was utilized as an additional AI assistant for content generation, research support, and validation of technical explanations used in portions of the project documentation.

Wondershare EdrawMax AI [9] was used to assist in the creation and refinement of project figures and diagrams. It supported the development of visual artifacts such as system design diagrams, architecture diagrams, use case diagrams, and other graphical representations included in the report.

Table 7.1 AI tools used in this project

AI Tool	Purpose of Use	Website
ChatGPT (OpenAI)	Academic writing improvement, report refinement, content organization, mockup documentation, and documentation support	https://chat.openai.com/
Claude (Anthropic)	Code assistance, debugging support, code refinement, and programming guidance	https://claude.ai/
Gemini (Google)	Content generation, technical research support, and validation of explanations	https://gemini.google.com/
Wondershare EdrawMax AI	Creation and refinement of diagrams, figures, and visual project documentation	https://www.edrawmax.com/

The use of these AI tools contributed to improved productivity, enhanced documentation quality, and more effective presentation of the project while maintaining human oversight throughout the development and reporting process.

CHAPTER 8

IMPLEMENTATION

8.1 Development Environment

The project was implemented using Python [3] and Flask [2] on the backend, HTML/CSS/JavaScript on the frontend, and JSON files for persistence in the deployed version.

8.2 Technology Stack

Table 8.1 *Technology Stack*

Technology	Use in GSMS
Python	Core programming language used for backend logic.
Flask	Web framework used for routing, sessions, and template rendering.
HTML/CSS/JavaScript	Frontend structure, styling, and client-side behavior.
Bootstrap	Responsive layout and interface styling.
JSON files	Lightweight persistence for products, users, and orders.
Gunicorn	Production WSGI server listed for deployment.
Render	Public hosting platform used for deployment.

8.3 Key Modules and Responsibilities

Table 8.2 *Key Modules and Responsibilities*

Module/File	Language/Tech	Responsibility
server.py	Python/Flask	Main application file; defines routes, sessions, and authentication.
data_store.py	Python	JSON-based data access layer for products, users, UOMs, and orders.
products_dao.py	Python	Data access object for product operations.
orders_dao.py	Python	Data access object for order operations.
uom_dao.py	Python	Data access object for units of measure.
schema.sql	SQL	MySQL schema for the relational version of the data model.
/ui/.html	HTML/Bootstrap	Templates for dashboard, login, registration, products, and orders.
/ui/css/.css	CSS	Styling and layout rules.
/ui/js/custom/.js	JavaScript/jQuery	Client-side interactivity and form support.

8.4 Sample Data Analysis

The sample store data contains eight units of measure, nine products, and four example orders. These sample records were sufficient to demonstrate dashboard output, product listing, and order history functionality.

- Products include items such as Rice, Milk, Eggs, and Soap.
- Orders include line-item details with quantity and computed totals.
- The sample data helps verify that the application behaves correctly with non-empty datasets.

CHAPTER 9

TESTING AND EVALUATION

9.1 Testing Strategy

The project was tested manually by exercising the major user flows: authentication, product CRUD actions, and order creation. Each test was checked against the stated requirements.

9.2 Test Cases and Results

Table 9.1 *Test cases and results*

Test Case	Expected Result	Observed Result
Valid login	User reaches dashboard	PASS
Invalid login	Access denied with error message	PASS
Add product	New product appears in the list	PASS
Edit product	Updated details are saved	PASS
Delete product	Product is removed from the listing	PASS
Create order	Order total is calculated and saved	PASS
View orders	Previously stored orders are visible	PASS

9.3 Evaluation

The evaluation confirms that the implemented application fulfills the core functional requirements and provides a stable workflow for the intended small-scale use case.

CHAPTER 10

DEPLOYMENT AND SECURITY

10.1 Deployment on Render

The project was run using Render's public hosting environment [5]. Deployment required the application to be configured for a production web server and an externally accessible URL.

10.2 Authentication and Password Security

User authentication is required before administrative pages can be accessed. Passwords are not stored in plain text; instead, the project uses salted SHA-256 hashing and stores the resulting hash in the user data file.

10.3 Input Validation and Session Handling

The application validates required inputs before processing product and order forms. Flask sessions are used to maintain login state after successful authentication.

10.4 Usability and Accessibility

Bootstrap-based [4] layout choices make the interface straightforward on desktop and tablet screens. The current implementation is functional and readable, although further accessibility auditing would improve the design.

10.5 Limitations and Security Considerations

- JSON persistence is simple but less scalable than a full database.
- Concurrency control is limited for simultaneous write operations.
- CSRF protection and advanced rate limiting are not implemented.
- Future production deployment should use HTTPS and stronger secret management.

CHAPTER 11

PROJECT MANAGEMENT

11.1 Project Planning

The Excel workbook includes a project schedule with planning, implementation, testing, and documentation tasks. This helped the team track progress and maintain a clear development sequence.

11.2 Team Responsibilities

The project was developed by a small student team. Responsibilities were divided across analysis, interface design, data handling, and documentation tasks.

11.3 Challenges Faced

- Aligning the implemented system with the requirement sheets.
- Maintaining consistency between the design schema and JSON-based storage.
- Preparing the deployment configuration for public hosting.
- Preparing presentation materials and final documentation.
- Most project tasks were completed by a single team member despite being a group project, which increased the workload, extended development time, and created additional project management challenges.

11.4 Gantt Chart Analysis

A project schedule was developed to monitor the progress of the Grocery Store Management System (GSMS) throughout its development lifecycle. The project followed a phased software development approach consisting of requirements analysis, system design, implementation, testing, deployment, and documentation. The Gantt chart was used to allocate responsibilities among team members, track milestones, and ensure timely completion of project deliverables.

Table 11.1 Project Schedule Summary

Phase	Activity	Duration	Status
Phase 1	Project Topic Selection and Planning	Week 1	Completed
Phase 2	Requirements Gathering and Analysis	Week 2	Completed
Phase 3	Literature Review and Research	Week 2 – Week 3	Completed
Phase 4	System Design (Architecture, UML, ERD)	Week 3 – Week 4	Completed
Phase 5	Database Design and Data Modeling	Week 4	Completed
Phase 6	User Interface Design	Week 5	Completed
Phase 7	Backend Development (Flask Application)	Week 5 – Week 7	Completed
Phase 8	Product Management Module Development	Week 6 – Week 7	Completed
Phase 9	Order Management Module Development	Week 7 – Week 8	Completed
Phase 10	Authentication and Security Features	Week 8	Completed

Phase	Activity	Duration	Status
Phase 11	Integration and System Testing	Week 9	Completed
Phase 12	Bug Fixing and Optimization	Week 9 – Week 10	Completed
Phase 13	Deployment on Render Platform	Week 10	Completed
Phase 14	Report Writing and Documentation	Week 10 – Week 12	Completed
Phase 15	Presentation Preparation and Final Review	Week 12	Completed

Project Timeline Overview:

Task ID	Task Name	Assigned Member	Duration	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Status
1	Project Planning & Requirement Study	All Team	1 Week	Completed										Completed
2	System Architecture Design	2677517	1 Week	Completed										Completed
3	Database Design (MySQL)	2677527	1 Week		Completed									Completed
4	Frontend UI Design	2677527	2 Weeks		Completed	Completed								Completed
5	Landing Page Development	2677529	1 Week			Completed							Completed	Completed
6	Product Management Module	2677527	2 Weeks			Completed	Completed						Completed	Completed
7	Category Management Module	2677527	1 Week				Completed						Completed	Completed
8	Order Management Module	2677527	2 Weeks				Completed	Completed					Completed	Completed
9	Flask Backend API Development	2677527	2 Weeks				Completed	Completed					Completed	Completed
10	Database Integration	2677527	1 Week					Completed					Completed	Completed
11	Frontend & Backend Integration	2677527	1 Week						Completed				Completed	Completed
12	Testing & Bug Fixing	All Team	2 Weeks						Completed	Completed			Completed	Completed
13	Documentation & Diagrams	2677530	2 Weeks							Completed	Completed		Completed	Completed
14	Mockup & UI Improvements	2677527	1 Week								Completed		Completed	Completed
15	Presentation Slides Preparation	2677527	1 Week									Completed	Completed	Completed
16	Final Review & Rehearsal	All Team	1 Week										Completed	Completed
17	Final Submission	All Team	1 Day										Completed	Completed

Figure 11.1 Screenshot of the Gantt Chart

The project was completed according to the planned schedule. Most development activities were completed within the allocated timeframe, while testing and documentation were performed iteratively throughout the later stages of the project. The use of a Gantt chart facilitated effective project monitoring, task allocation, and milestone tracking, contributing to the successful completion of the system.

11.5 Screenshot of the Requirements Traceability Matrix (RTM)

The Requirements Traceability Matrix (RTM) was developed to establish a clear relationship between project requirements, system design components, implementation modules, and testing activities. The matrix ensures that every functional and non-functional requirement is properly addressed during development and verified through testing. By mapping requirements to their corresponding implementation and validation methods, the RTM improves project accountability, helps identify missing requirements, and confirms that the final system satisfies the specified objectives. It serves as an important project management and quality assurance artifact throughout the software development process.

Req. ID	Requirement Description	Requirement Type	Priority	Use Case ID	Module	Database Entity	Test Case ID	Acceptance Criteria	Status
FR001	View Products	Functional	High	UC001	Product Management	Product	TC001	Product list displays successfully	Implemented
FR002	Add Product	Functional	High	UC002	Product Management	Product	TC002	New product saved and displayed	Implemented
FR003	Edit Product	Functional	High	UC003	Product Management	Product	TC003	Product information updated correctly	Implemented
FR004	Delete Product	Functional	High	UC004	Product Management	Product	TC004	Product removed successfully	Implemented
FR005	Add Category	Functional	High	UC006	Category Management	Category	TC006	Category saved successfully	Implemented
FR006	Create Order	Functional	High	UC007	Order Management	Order, Order_Details	TC007	Order record created	Implemented
FR007	Calculate Order Total	Functional	High	UC007	Order Management	Order_Details	TC008	Correct total calculated	Implemented
FR008	Store Order Details	Functional	High	UC007	Order Management	Order, Order_Details	TC009	Order data persists in database	Implemented
FR009	View Orders	Functional	High	UC008	Order Management	Order	TC010	Order records retrievable	Implemented
FR010	User Authentication	Functional	High	UC009	Authentication Module	User	TC011	System authenticates users with valid credentials	Implemented
FR011	Secure Logout	Functional	High	UC010	Authentication Module	User Session	TC012	User session is terminated successfully	Implemented
FR012	Input Validation	Functional	High	UC013	Frontend & Backend Validation	All Entities	TC016	Invalid or incomplete input is rejected with appropriate error messages	Implemented
FR013	Database Connectivity	Functional	High	UC014	Database Layer	All Entities	TC017	Application successfully connects to the database	Implemented
FR014	CRUD API Operations	Functional	High	UC015	REST API Layer	Product, Category, Order	TC018	Create, Read, Update, and Delete API endpoints work as expected	Implemented
NFR001	Response time under 3 seconds	Non-Functional	High	N/A	Backend/API	All Modules	TC011	95% requests under 3 sec	Verified
NFR002	Data consistency and reliability	Non-Functional	High	N/A	Database	All Entities	TC012	No inconsistent records	Verified
NFR003	Ease of use	Non-Functional	Medium	N/A	Frontend UI	All Modules	TC013	Users complete tasks unaided	Verified
NFR004	Maintainable modular architecture	Non-Functional	Medium	N/A	Frontend + Backend	All Modules	TC014	Modules independently maintainable	Verified
NFR005	System availability	Non-Functional	Medium	N/A	Deployment	Entire System	TC015	Availability >=95%	Verified
NFR006	Scalability	Non-Functional	Medium	N/A	Database + Backend	All Entities	TC016	Performance acceptable with growth	Verified
NFR007	Input validation and security	Non-Functional	High	N/A	Backend/API	All Entities	TC017	Invalid inputs rejected	Verified
NFR008	Data integrity	Non-Functional	High	N/A	Database	All Entities	TC018	Referential integrity maintained	Verified
NFR009	Portability	Non-Functional	Low	N/A	Deployment	Entire System	TC019	Runs on supported environments	Verified
NFR010	Recoverability	Non-Functional	Medium	N/A	Database	All Entities	TC020	Backup restoration successful	Verified

Figure 11.2 Screenshot of the Requirements Traceability Matrix (RTM)

CHAPTER 12

CONCLUSION AND FUTURE WORK

12.1 Conclusion

The Grocery Store Management System provides a practical web-based solution for managing products, units of measure, and orders. The project demonstrates a complete academic software lifecycle from requirements analysis through deployment and evaluation.

12.2 Future Enhancements

- Replace JSON storage with a relational database for stronger scalability.
- Add reporting and analytics features for sales summaries.
- Improve authentication with stronger session and CSRF protection.
- Expand the interface with richer dashboard visuals and accessibility improvements.
- Add automated unit and integration testing.

REFERENCES

- [1] GitHub Repository: Grocery Store Management System
- [2] Flask Documentation
- [3] Python Documentation
- [4] Bootstrap Documentation
- [5] Render Documentation
- [6] ChatGPT AI
- [7] Claude AI
- [8] Google Gemini AI
- [9] Wondershare EdrawMax AI

APPENDICES

Appendix A: Installation Guide

This appendix provides the steps required to install and run the Grocery Store Management System. The source code of the project is uploaded in GitHub Repository: Grocery Store Management System [1].

System Requirements

- Python 3.10 or higher
- Internet connection (for package installation)
- Modern web browser (Google Chrome, Microsoft Edge, Firefox)

Installation Steps

1. Download or clone the project repository.
2. Navigate to the project directory.
3. Install the required dependencies:

```
pip install -r requirements.txt
```

4. Start the Flask application:

```
python server.py
```

5. Open the browser and navigate to:

```
http://127.0.0.1:5000
```

6. Log in using a valid account to access the system dashboard.

Appendix B: User Manual

User Login

4. Open the application.
5. Enter a valid email and password.
6. Click the Login button.

Product Management

7. Navigate to the Product Management page.
8. Add new products using the product form.
9. Edit existing products when required.
10. Delete products that are no longer needed.

Order Management

11. Navigate to the Orders page.
12. Create a new customer order.
13. Select products and quantities.
14. Review and save the order.
15. View previously created orders.

User Logout

16. Click the Logout option from the navigation menu.
17. The session will be terminated securely.

Appendix C: Source Code Structure

The project follows a modular architecture to improve maintainability and scalability.

Backend Components

- server.py – Main Flask application and route definitions.
- data_store.py – JSON data management layer.
- products_dao.py – Product-related operations.
- orders_dao.py – Order-related operations.
- uom_dao.py – Unit-of-measure operations.

Frontend Components

- login.html
- register.html
- manage-product.html
- order.html
- index.html

Static Assets

- CSS files
- JavaScript files
- Images and icons

The modular organization separates business logic, data access, and user interface components.

Appendix D: Gantt Chart

Figure D.1 Project Development Schedule

Task ID	Task Name	Assigned To	Duration	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Status
1	Project Init	All Team	1 Week	Completed										Completed
2	System Design	2677527	1 Week	Completed										Completed
3	Database Design	2677527	1 Week		Completed									Completed
4	Frontend Development	2677527	2 Weeks		Completed	Completed								Completed
5	Backend Development	2677527	1 Week			Completed								Completed
6	Product Integration	2677527	2 Weeks			Completed	Completed							Completed
7	Category Management	2677527	1 Week				Completed							Completed
8	Order Management	2677527	2 Weeks				Completed	Completed						Completed
9	Flask API Development	2677527	2 Weeks				Completed	Completed						Completed
10	Database Integration	2677527	1 Week					Completed						Completed
11	Frontend Integration	2677527	1 Week						Completed					Completed
12	Testing	2677527	2 Weeks						Completed	Completed				Completed
13	Documentation	2677527	2 Weeks							Completed	Completed			Completed
14	Mockups	2677527	1 Week								Completed			Completed
15	Presentation	2677527	1 Week									Completed		Completed
16	Final Review	2677527	1 Week											Completed
17	Final Submission	All Team	1 Day											Completed

The Gantt chart illustrates the project timeline, development phases, task dependencies, and completion status throughout the project lifecycle.

Appendix E: Requirements Traceability Matrix (RTM)

Table E.1 Requirements Traceability Matrix

Req. ID	Requirement	Requirementer	Priority	Use Case	Module	Database	Test Case	Acceptance	Status
FR001	View Products	Functional	High	UC001	Product	Product	TC001	Product list	Impleme
FR002	Add Product	Functional	High	UC002	Product	Product	TC002	New product	Impleme
FR003	Edit Product	Functional	High	UC003	Product	Product	TC003	Product info	Impleme
FR004	Delete Product	Functional	High	UC004	Product	Product	TC004	Product ren	Impleme
FR005	Add Category	Functional	High	UC006	Category	Category	TC006	Category sa	Impleme
FR006	Create Order	Functional	High	UC007	Order M	Order, Or	TC007	Order recor	Impleme
FR007	Calculate Order	Functional	High	UC007	Order M	Order_De	TC008	Correct tota	Impleme
FR008	Store Order D	Functional	High	UC007	Order M	Order, Or	TC009	Order data	Impleme
FR009	View Orders	Functional	High	UC008	Order M	Order	TC010	Order recor	Impleme
FR010	User Authent	Functional	High	UC009	Authent	User	TC011	System aut	Impleme
FR011	Secure Logout	Functional	High	UC010	Authent	User Sess	TC012	User sessio	Impleme
FR012	Input Validati	Functional	High	UC013	Fronten	All Entitie	TC016	Invalid or in	Impleme
FR013	Database Con	Functional	High	UC014	Databas	All Entitie	TC017	Application	Impleme
FR014	CRUD API Ope	Functional	High	UC015	REST AP	Product, C	TC018	Create, Rea	Impleme
NFR001	Response tim	Non-Function	High	N/A	Backend	All Modul	TC011	95% request	Verified
NFR002	Data consisten	Non-Function	High	N/A	Databas	All Entitie	TC012	No inconsis	Verified
NFR003	Ease of use	Non-Function	Medium	N/A	Fronten	All Modul	TC013	Users comp	Verified
NFR004	Maintainable	Non-Function	Medium	N/A	Fronten	All Modul	TC014	Modules inc	Verified
NFR005	System availa	Non-Function	Medium	N/A	Deployn	Entire Sys	TC015	Availability	Verified
NFR006	Scalability	Non-Function	Medium	N/A	Databas	All Entitie	TC016	Performanc	Verified
NFR007	Input validati	Non-Function	High	N/A	Backend	All Entitie	TC017	Invalid inpu	Verified
NFR008	Data integrity	Non-Function	High	N/A	Databas	All Entitie	TC018	Referential	Verified
NFR009	Portability	Non-Function	Low	N/A	Deployn	Entire Sys	TC019	Runs on sup	Verified
NFR010	Recoverability	Non-Function	Medium	N/A	Databas	All Entitie	TC020	Backup rest	Verified

The RTM establishes traceability between project requirements, system design components, implementation modules, and testing activities. It ensures that all specified functional and non-functional requirements have been addressed and verified during development.

Appendix F: Functional and Non-Functional Requirements

F.1 Functional Requirements

The Functional Requirements define the core services and operations that the Grocery Store Management System must provide.

Table F.1 Functional Requirements

Require	Functional Req	Priority	Rationale	Acceptance Crite	Assigned Team Member ID
FR001	View Products	High	Users must b	Product list disp	2677527
FR002	Add Product	High	Admin must	New product is	2677527
FR003	Edit Product	High	Product infor	Updated inform	2677527
FR004	Delete Product	High	Obsolete pro	Product no long	2677527
FR005	Add Category	High	New product	Category is save	2677527
FR006	Create Order	High	Customers m	Order record is c	2677527
FR007	Calculate Orde	High	Order value r	Correct total is c	2677527
FR008	Store Order De	High	Order history	Order data pers	2677527
FR009	View Orders	High	Admin must	Order records ai	2677527
FR10	User Authentic	High	Restricts acc	System authent	2677517
FR11	Secure Logout	Medium	Ensures sessi	User session is t	2677530
FR12	Input Validatio	High	Prevents incor	Invalid input is r	2677529
FR13	Database Conn	High	Supports per	Application esta	2677527
FR14	CRUD API Oper	High	Enables stand	All Create, Read	2677527

The implemented system satisfies all high-priority functional requirements. Product management, order processing, and authentication functionalities were successfully developed and tested.

F.2 Non-Functional Requirements

The Non-Functional Requirements describe the quality attributes and operational constraints of the system.

Table F.1 Non-Functional Requirements

Requireme	Category	Requirement	Priority	Rationale	Acceptance Crite	Assigned Team Member ID
NFR001	Performance	Response time ur	High	Provides acc	95% of request	2677527
NFR002	Reliability	System maintains	High	Prevents dat	No inconsisten	2677527
NFR003	Usability	Interface should k	Medium	Reduces trai	Users complet	2677527
NFR004	Maintainabili	Code should be m	Medium	Simplifies fu	Modules can b	2677527
NFR005	Availability	System should be	Medium	Supports der	Availability >=	2677527
NFR006	Scalability	System should su	Medium	Supports fut	Performance r	2677530
NFR007	Security	Inputs must be va	High	Protects dat	Invalid input is	2677527
NFR008	Data Integrity	Database relation	High	Ensures accu	Referential int	2677527
NFR009	Portability	System should ru	Low	Improves de	Runs on suppo	2677529
NFR010	Recoverabilit	Database backup	Medium	Reduces risk	Backup restora	2677517

The system achieves the specified quality objectives through a modular Flask architecture, Bootstrap-based user interface [4], password hashing, session management, and cloud deployment support.